Tameng — Technical Documentation

2026 Shenzhen International FinTech Competition (FinTechathon) — International Track, Topic F: Agent Wallet Universitas Diponegoro, Indonesia

Tameng is a mobile AI wallet agent that lets an Indonesian migrant worker set an auto-send mandate once, and then sends USDC home on schedule without the owner being online. The LLM decides *what the owner wants*; code decides *whether it is allowed*. This document describes the architecture, the core algorithms, the security design, the stack, how to deploy it, how it was tested, and what is still limited.

---

1. Overview

1.1 Problem

A migrant worker sends money home every month. Existing remittance apps charge an average of **5.13%** for a US$200 transfer to Indonesia (World Bank Remittance Prices Worldwide, Q3 2025) and still require the worker to be online, in the app, at the moment of sending. A wallet agent could remove that manual step, but an agent that can sign transactions is also a target: prompt injection, a forwarded message from a compromised family account, a look-alike address, or a "support" call can all try to turn the agent into an attacker's signer.

The design goal is therefore two-sided:

- **Autonomy:** the agent sends on schedule, to several recipients, and keeps a savings buffer.
- **Containment:** no message, tool output, forwarded text or changed setting can push a transfer outside the owner's mandate.

1.2 Solution

The owner uses the Expo mobile app to manage contacts, set limits, and create an **auto-send** (mandate) with 1–5 recipients, split amounts, a schedule, and a minimum balance. The backend runs a scheduler. Every transfer — scheduled or one-off — passes the same policy pipeline immediately before signing. If the pipeline denies, nothing is signed and the owner gets a denied card. If it requires confirmation, the transfer waits for the owner's PIN.

1.3 Repository layout

```
app/       Expo / React Native app (Home, Chat, Auto-send, Activity, Contacts, Settings)
server/    HTTP API, scheduler, storage — wires agent + policy + onchain together
agent/     Message -> structured intent (rule-based today, LLM-ready interface)
policy/    checkPolicy(), detectInjection(), provenance, intent lock, lookalike, fixed limits
onchain/   Agent wallet, USDC transfer, balances, receipts, tx-hash evidence
docs/      Technical doc, security report, execution evidence, deck
```

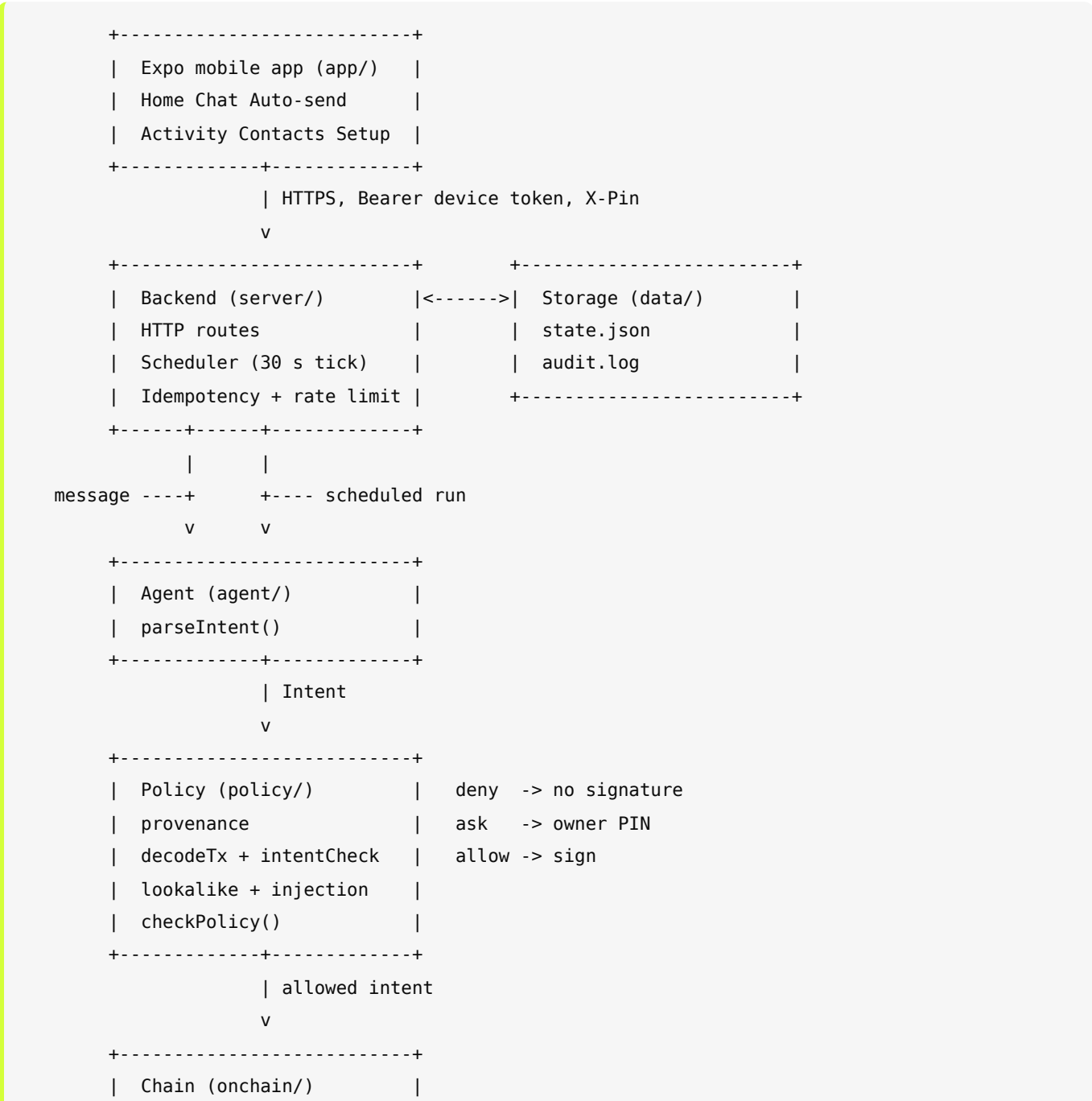
1.4 Runtime shape

LAYER	TECHNOLOGY	PROCESS
Mobile	Expo SDK 57 / React Native, TypeScript	Expo Go or Android build
Backend	Node.js ≥ 22.18, TypeScript (no build step)	`npm run server`
Agent	Rule-based parser today; same interface is LLM-ready	In-process
Policy	Pure TypeScript, compiled-in limits	In-process
Chain	Base Sepolia, USDC, viem	RPC `https://sepolia.base.org`
Storage	`data/state.json` + append-only `data/audit.log`	Local disk

---

2. System architecture

2.1 Component diagram



```
        |  viem wallet client       |
        |  USDC transfer + receipt  |
        +-------------+-------------+
                      |
                      v
            Base Sepolia (USDC)
```

2.2 Trust boundaries

BOUNDARY	TRUSTED	UNTRUSTED
Mobile app ↔ backend	device token + PIN hash comparison server-side	the request body itself
Backend ↔ agent	parsed intent shape	raw message text, tool output, forwarded text
Backend ↔ chain	allowlisted recipient, decoded calldata	anything the agent proposes
Backend ↔ storage	server process	files on disk are clamped on read (`clampConfig`)

The agent **never** holds a key and never signs. `server/` calls `policy/` again immediately before `onchain/` signs, so a decision made a minute ago cannot outrun a limit lowered now.

2.3 Data flow: chat transfer

1. App sends `POST /chat` with `Authorization: Bearer <device token>`.
2. `agent/parseIntent.ts` extracts `{kind, amount, address, nameHint, injection, addressConfusable}`.
3. `server/core.ts` resolves the recipient (exact contact, confusable address, or nobody).
4. `policy/*` produces `allow`, `confirm`, or `deny` with plain-language reasons.
5. allow → `onchain.transfer()` broadcasts and returns a `TxRef` (hash + explorer URL).
6. confirm → the intent is stored for 10 minutes; `POST /confirm/:intentId` re-runs policy and requires `X-Pin`.
7. deny → the decision card is stored in activity and audit; no signature happens.

2.4 Data flow: scheduled mandate

```
scheduler tick (30 s)
  -> for each mandate with status active|postponed and nextRunAt <= now
     -> deterministic runId = "run-<mandate>-<scheduledAt>"
     -> if wallet balance - total < minBalance  => postponement path (no tx)
     -> for each recipient:
          -> executeTransfer(contact, amount, "mandate", {runId, key})
          -> policy gate -> onchain.transfer()
          -> store hash under run.transfers[key]
     -> lastRun.txs = hashes; nextRunAt = next slot
```

---

3. Core algorithms

3.1 Intent extraction (`agent/parseIntent.ts`)

`parseIntent(message)` returns a small, auditable shape:

```
type IntentKind = 'send' | 'mandate' | 'blocked_report' | 'help';

interface ParsedIntent {
  kind: IntentKind;
  amount: number | null;
  address: string | null;
  nameHint: string | null;
  injection: string[];
  addressConfusable: boolean;
  text: string;
}
```

The parser is intentionally **rule-based** in this build: keyword classes for send/mandate, one amount, one address, a name hint, and the injection matches from `policy/detectInjection.ts`. It folds confusable characters in a typed address first (`0x0O47…` → `0x0047…`) so the trap is reported as a look-alike instead of silently becoming "unknown contact". An LLM can replace the parser later behind the same interface; the policy layer does not depend on the parser being correct, only on the decoded transaction being checked.

3.2 Mandate execution (`server/core.ts`)

A mandate is:

```
interface MandateInput {
  recipients: { contactId: string; amount: string }[]; // 1–5, may include "savings"
  schedule: { type: 'monthly' | 'interval'; dayOfMonth?: number; minutes?: number };
  minBalance: string;
}
```

Properties:

- **Unattended.** `runDueMandates()` runs on a 30-second tick; no owner input is required.
- **Atomic run identity.** `runId = "run-" + mandateId + "-" + Date.parse(scheduledAt)`. The id is deterministic, so a crash/restart on the same slot resumes the same run instead of creating a second one.
- **Per-recipient idempotency.** Each transfer is keyed `mandate:<runId>:<index>` and its hash is persisted **before** the balance moves. A replay finds the cached `TxRef` and does not broadcast again.
- **Postponement.** If `balance - total < minBalance`, the run is marked `postponed`, no transaction is sent, `consecutivePostpones` increments, and the next attempt is scheduled three days later. After three consecutive postponements the mandate auto-pauses and the owner is notified.
- **Partial failure.** If one recipient fails, the run continues with the rest; the run is marked `failed`, the remaining recipients are retried on the next scheduled run, and the already-sent recipients are skipped through their cached hashes.

- **Receipt reconciliation.** `settlePending()` asks the RPC for every pending hash and moves it to `confirmed`, `failed`, or `unknown`. A missing receipt stays `pending`, never `failed`.

3.3 Intent lock (`onchain/decodeTx.ts` + `policy/intentCheck.ts`)

The intent lock closes the gap between "the agent says transfer 10 USDC to Ibu" and "the calldata that will actually be signed". The function selector is decoded and compared:

SELECTOR	FUNCTION	RULE
`0xa9059cbb`	`transfer(address,uint256)`	token, recipient and amount must match the intent
`0x095ea7b3`	`approve(address,uint256)`	always requires owner confirmation; never unlimited by default
`0x39509351`	`increaseAllowance(address,uint256)`	always requires owner confirmation
`0xd505accf`	`permit(...)`	treated as an approval, never autonomous
anything else	—	denied

The decoded recipient and amount are compared to the intent **before** the signer is called. A mismatch is a hard `deny` (`INTENT_MISMATCH`), even if the policy limits would have allowed the amount. This is what stops a compromised or hallucinating agent from signing a different transfer than the one the owner approved.

3.4 Provenance guard (`policy/provenance.ts`)

Every message is labelled before the parser sees it:

SOURCE	LABEL	CAN TRIGGER A TRANSFER?
The owner typing in the app	`USER`	yes, within policy
Mandate scheduler	`MANDATE`	yes, within policy
Tool output, forwarded message, decoded text, token metadata	`EXTERNAL`	no — `EXTERNAL_PROVENANCE` deny

The API accepts a `source` field; anything other than `"user"` is treated as `EXTERNAL`. The guard also reports the excerpt that tried to look like an order so the activity card can show the owner exactly what was ignored.

3.5 Injection detection (`policy/detectInjection.ts`)

A defence-in-depth layer that normalises first, then scans:

1. **Normalise:** strip invisible characters, fold Cyrillic/Greek homoglyphs, NFKC-normalise, collapse "s p a c e d o u t" runs.
2. **Decode:** scan base64 blobs (16+ printable chars) and Morse runs, then rescan the decoded text as if it had been written in the open.
3. **Match:** instruction override, role escalation, secret extraction, drain attempt, policy bypass, fake authority, urgency pressure — in English and Indonesian.
4. **Tool output:** an imperative with an address/amount inside tool output is suspicious even when it is phrased politely.

Whatever this layer misses is still blocked by `checkPolicy()`: injection detection improves the explanation, not the enforcement boundary.

3.6 Look-alike detection (`policy/lookalike.ts`)

The owner is warned when an address resembles a saved contact:

KIND	RULE
`same-short-form`	same first 6 and last 4 characters as a contact
`near-identical`	edit distance ≤ 2 from a contact
`confusable`	identical after folding `0/o`, `1/l/i`, `5/s`, `8/b`
`shared-prefix` / `shared-suffix`	same short prefix or suffix as a contact

An exact match of a saved contact is never a look-alike. A typed address that is not valid hex is folded back to hex first, so `0x0O47…` is reported as a confusable imitation of `Adik`, not as a random unknown address.

3.7 Policy engine (`policy/checkPolicy.ts`, `policy/permissions.ts`)

`checkPolicy(tx, config, state)` is the last gate before signing:

```
deny if:  agent paused
          chain not allowed (base-sepolia only)
          action not allowed (transfer only)
          malformed recipient address
          amount not a positive number
          recipient not in allowlist
          amount > perTxLimit
          spentToday + amount > dailyLimit

confirm if: amount > confirmAbove
            action is an approval

allow otherwise
```

Limits are owner-editable but live inside fixed ceilings compiled into `policy/permissions.ts`: max 500 USDC per transfer, max 1,000 USDC per day, max confirm threshold 100 USDC, and a minimum buffer of 1 USDC. `clampConfig()` reapplies those bounds when settings are read from disk, so hand-editing `state.json` cannot widen them. Settings changes that try to exceed the ceilings are rejected with a ready-to-show reason.

---

4. Security design

4.1 Permission tiers

TIER	DECISION	EXAMPLES
Autonomous	`allow`	mandate run within limits to a verified contact; small one-off transfer to a contact

TIER	DECISION	EXAMPLES
Owner confirmation	`confirm`	one-off transfer above the confirm threshold; any token approval
Blocked	`deny`	unknown or look-alike address, over limit, wrong chain, wrong action, paused agent, external-text trigger, injection

4.2 Threat model and mitigations

THREAT	MITIGATION
Prompt injection in the owner's own message	`detectInjection` + `checkPolicy`; drain phrases still need an allowlisted recipient and a limit
Injection hidden in tool output, a memo, or forwarded text	provenance `EXTERNAL`; external text can never trigger a transfer
Look-alike address	`findLookalike` + confusable folding; the transfer is denied and the owner sees which contact it imitates
Stolen or guessed PIN	PIN hash only, 5 attempts then 15-minute lock; all sensitive endpoints require `X-Pin`
Stolen device token	token is per-deployment and server-side; production deployments must set `DEVICE_TOKEN` and `APP_PIN_HASH`
Compromised agent output	intent lock compares decoded calldata to the intent before signing; approvals always require the owner
Limit widening through chat or settings	fixed ceilings compiled into code; chat has no settings write path
Lost RPC / duplicate broadcast	retry once with the same nonce when no hash exists; per-run and per-recipient idempotency keys
Crash mid-run	deterministic `runId`; hashes persisted before balance changes; terminal runs are repaired, never replayed
Key theft from the repo	private key only in `.env` on the backend machine, gitignored; the mobile app has no key material

4.3 Red-team results and benchmark

The full results are in `SECURITY_REPORT.md`. Summary:

- Six targeted attacks (instruction override, external provenance, look-alike address, private key extraction, limit raise through chat, mandate address change) all produced **deny** or a clarify with no settings change and no transaction.
- `POST /benchmark/run` runs 54 deterministic cases (49 attack + 5 benign) on a state snapshot and restores it afterwards, forcing simulation even on a live server. Result on 2026-10-19: **49/49 attack cases blocked, 0 attacks moved funds, 0 failures.**

4.4 Audit and evidence

- `data/audit.log` is append-only: one JSON line per transfer, blocked attempt, PIN failure, auto-send run, postponement, and settings change.
- `docs/EXECUTION_EVIDENCE.md` holds real Base Sepolia hashes with date, scenario, policy decision, amount, and an explorer link. In live mode `server/evidence.ts` appends the row

automatically; simulated transfers, tests, and the benchmark never write there.

- The evidence file is the source for the submission PDF; the atomic-write flow means a row only appears after a broadcast actually returned a hash.

---

5. Tech stack

LAYER	CHOICE	WHY
Mobile	Expo SDK 57, React Native, Expo Router, TypeScript	one codebase for Android + web fallback, runs in Expo Go for demos
Backend	Node.js ≥ 22.18, TypeScript run directly	no build step, fast iteration during a 48-hour event
HTTP	Node `http` module, hand-rolled router	small surface, easy to audit, no framework lock-in
Chain	viem 2.x	typed contracts, nonce control, receipt handling
Network	Base Sepolia, USDC (6 decimals)	cheap testnet, official Circle USDC available, EVM tooling
Agent	rule-based parser now, LLM-ready interface	policy is the security boundary; the parser can be swapped
Storage	JSON file + append-only log	zero-ops demo storage; easy to inspect and back up
Tests	Node built-in `node --test`	runs TypeScript directly, no extra toolchain
PDFs	Markdown → styled HTML → headless Chrome `--print-to-pdf`	reproduces submission documents from the repo

---

6. Deployment guide

6.1 Backend

```
# requirements: Node.js >= 22.18
cp .env.example .env

# .env (never commit)
# AGENT_WALLET_PRIVATE_KEY=0x...      # testnet-only agent wallet
# DEVICE_TOKEN=<random>               # app auth
# APP_PIN_HASH=<sha256 of the PIN>    # node -e "console.log(require('crypto').createHash('sha256')
# TAMENG_LIVE=1                       # 0 = simulated hashes, 1 = broadcast
# USDC_CONTRACT_ADDRESS=0x036CbD...   # Base Sepolia USDC
# SAVINGS_WALLET_ADDRESS=0x...        # second owner wallet
# PORT=3000

npm install
npm test
npm run server
# [tameng] API on http://localhost:3000
# [tameng] phone: EXPO_PUBLIC_API_URL=http://192.168.x.x:3000
```

The server binds `0.0.0.0` so the phone on the same Wi-Fi can reach it. `/health` is the only endpoint without a device token and returns `{"ok":true,"mode":"live|simulated",...}`.

6.2 Mobile app

```
cd app
npm install
cp .env.example .env
# Sample data mode: leave EXPO_PUBLIC_API_URL empty.
# Live mode:
# EXPO_PUBLIC_API_URL=http://192.168.x.x:3000
# EXPO_PUBLIC_DEVICE_TOKEN=<same DEVICE_TOKEN>
npx expo start          # scan QR with Expo Go, or press w for web
npm run typecheck
npm run export:web      # static web fallback
```

6.3 Remote access (judges, mobile data)

```
npx cloudflared tunnel --url http://localhost:3000   # or: ngrok http 3000
# put the printed https:// URL into app/.env as EXPO_PUBLIC_API_URL, restart Expo with --clear
```

6.4 Reproduce the evidence locally

```
# 1. unit + integration suite
npm test                                  # 65/65 pass

# 2. attack benchmark (never moves real funds)
npm run server &                          # or use a running instance
curl -X POST -H 'Authorization: Bearer demo-device' http://localhost:3000/benchmark/run

# 3. live evidence (only with a funded testnet wallet and TAMENG_LIVE=1)
#    POST /chat "Send 40 USDC to Ibu" -> POST /confirm/:intentId with X-Pin: 123456
#    each broadcast appends a row to docs/EXECUTION_EVIDENCE.md
```

6.5 Security checklist before making the repo public

1. `git log --all -- .env` and `git grep -I '0x[0-9a-fA-F]{64}'` return nothing sensitive.
2. `.env` is untracked and ignored; `.env.example` contains no real key.
3. `DEVICE_TOKEN` and `APP_PIN_HASH` are set to non-demo values on any public host.
4. The wallet in the deployment holds testnet funds only.
5. `TAMENG_LIVE=1` is opt-in; a fresh clone runs simulated unless the operator flips it.

---

7. Testing

SUITE	WHAT IT PROVES	COUNT
`policy/policy.test.ts`	limits, allowlist, pause, chain/action checks, malformed input, approvals	13
`policy/security.test.ts`	injection (plain, Indonesian, base64, Morse, zero-width), look-alike, provenance, intent lock	29
`onchain/sendUsdc.test.ts`	calldata round-trip, simulated mode, live guard under test	2
`server/server.test.ts`	endpoints, PIN lock, idempotency, scheduler, postponement, auto-pause, evidence wiring	21
`POST /benchmark/run`	54 attack/benign prompts on a snapshot	54 cases

Run everything:

```
npm test
```

Result on the submitted build: **65/65 passing**. The benchmark result on the live build: **49/49 attack cases blocked, 0 moved funds**.

---

8. Limitations and future work

- **Rule-based parser.** `agent/parseIntent.ts` handles the demo vocabulary; the LLM tool-calling path is behind the same interface (`TODO(TMG-49)`) and is the first thing to finish after the event.
- **Demo token provenance.** The 11 recorded transfers use Aave's Base Sepolia faucet USDC (`0xba50Cd2A20f6DA35D788639E581bca8d0B5d4D5f`) because the Circle faucet limits 20 USDC per address per two hours. The official Circle USDC address is funded and configured as the default; the `SECURITY_REPORT.md` states the provenance explicitly.
- **Key custody.** The agent key lives in `.env`; production needs a managed signer or HSM with the same policy hook in front.
- **Storage.** JSON storage is single-process and fine for a demo; production wants a database with per-wallet access rules (see the Replica pack conventions in the docs folder).
- **Notifications.** Postponement and block events are in-app only; push or WhatsApp delivery would help a worker who is not looking at the app.
- **No independent audit.** The security results are a self-assessment plus deterministic regression tests, not a third-party penetration test.
- **Video and APK.** The demo video and Android build are submission artifacts handled by the mobile owner; the static web build (`app/dist`) is the fallback if the native build slips.

Appendix A — Environment variables

VARIABLE	DEFAULT	USED BY
`PORT`	`3000`	server
`DEVICE_TOKEN`	`demo-device`	server auth (`Authorization: Bearer`)

VARIABLE	DEFAULT	USED BY
`APP_PIN_HASH`	sha256(`123456`)	server PIN check
`RPC_URL`	`https://sepolia.base.org`	onchain
`CHAIN_ID`	`base-sepolia`	policy config
`USDC_CONTRACT_ADDRESS`	Circle Base Sepolia USDC	onchain
`AGENT_WALLET_PRIVATE_KEY`	—	onchain (testnet only)
`SAVINGS_WALLET_ADDRESS`	demo savings address	mandate recipient `savings`
`TAMENG_LIVE`	`0`	onchain live/simulated switch
`TAMENG_NO_PERSIST`	unset	server writes nothing to `data/` when `1`
`EXPO_PUBLIC_API_URL`	—	app (empty = sample data)
`EXPO_PUBLIC_DEVICE_TOKEN`	—	app auth

Appendix B — HTTP API

METHOD	PATH	AUTH	PURPOSE
`GET`	`/health`	none	liveness + mode
`GET`	`/wallet`	token	balances, savings progress, upcoming runs, protection report
`POST`	`/chat`	token	message → decision card / clarification / mandate draft
`POST`	`/confirm/:intentId`	token + PIN	approve a `confirm` card
`GET`	`/activity`	token	transfer and blocked-attempt feed
`POST`	`/activity/read`	token	clear the unread badge
`GET` / `POST`	`/mandates`	token (+ PIN on write)	list / create auto-send
`PATCH` / `DELETE`	`/mandates/:id`	token + PIN	pause, edit, delete
`GET` / `POST` / `DELETE`	`/contacts`	token (+ PIN on write)	allowlist
`GET` / `PATCH`	`/settings`	token (+ PIN except pause)	limits, demo mode, emergency pause
`POST`	`/benchmark/run`	token	54-case attack benchmark on a snapshot

Request/response shapes live in `API_CONTRACT.md`; sample responses in `../app/mocks/`.

Appendix C — Evidence summary (2026-10-19)

- Network: Base Sepolia (`base-sepolia` , chain id 84532), USDC 6 decimals.
- Agent wallet: `0x057BE625D711c504D36D2c4a59EC2f6C6B3ffdDC` .
- Savings wallet: `0x9C2ebEB50975FE0fe475Ed4Fee4c8E6761110fb5` .
- 11 successful transactions, all receipt status `success` , blocks 48324582–48325764.

- AC-01: unattended 60 + 20 + 20 USDC every 2 minutes, 3 hashes per run.
- AC-02: postponement with zero transactions at balance 730 < minBalance 800.
- AC-03: 40 USDC to Ibu with confirmation and explorer link.
- AC-08: address-change attempt denied, no mandate edited.
- Benchmark: 49/49 attack cases blocked, 0 funds moved.

Full rows and explorer links: `EXECUTION_EVIDENCE.md`.
